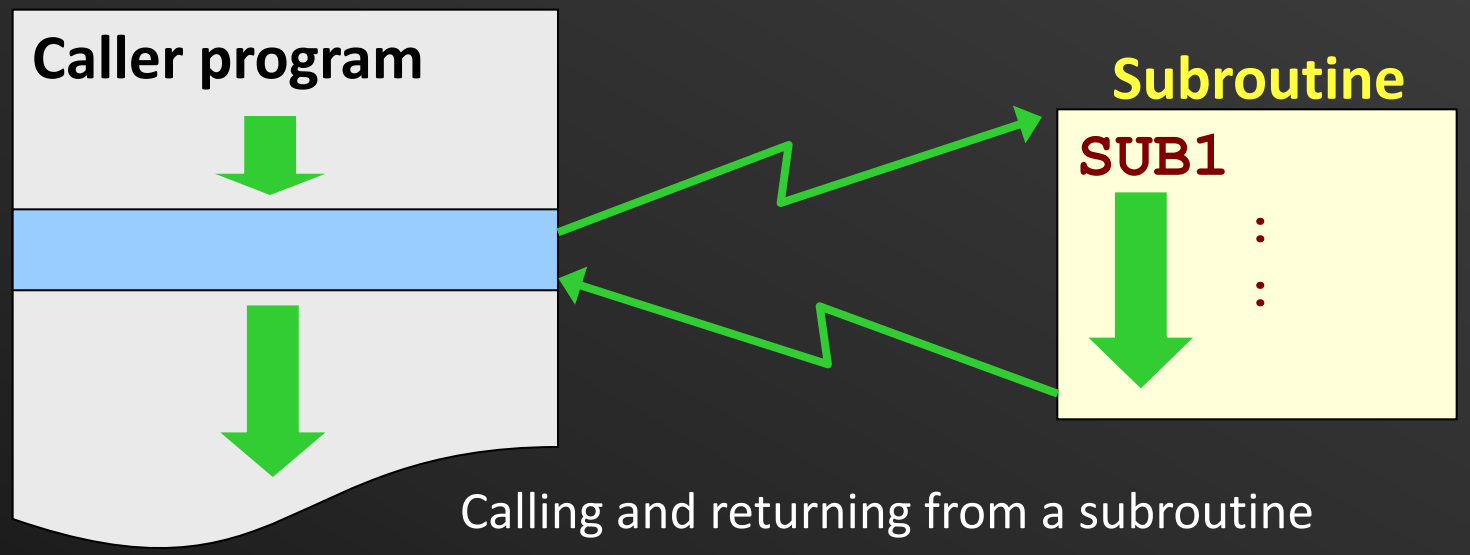
Chapter 6: Modular Programming Lecture 1 (live)

Mohamed M. Sabry Aly

N4-02c-92

Subroutines

- Modules (e.g., functions in C) are implemented as **subroutines**
- Subroutine can be called from various parts of the program
- Caller and callee
  - Caller: the program that calls subroutine (SUB1)
  - Callee: subroutine (SUB1)



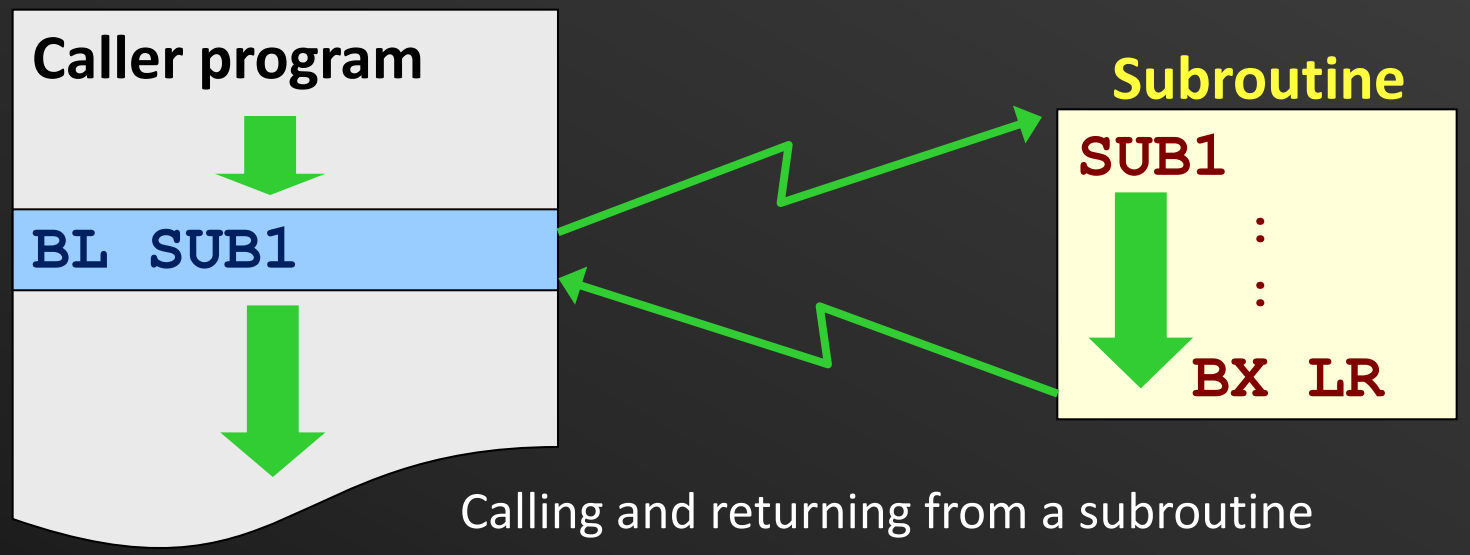
Calling and returning from a subroutine

Subroutines

- On completion, subroutine returns control to the caller
  - Exactly after the subroutine was called
- Calling and returning from a subroutine
  - To go to subroutine (SUB1): **`BL SUB1`**
  - To return to caller program: **`BX LR`**

`BL: branch with link`

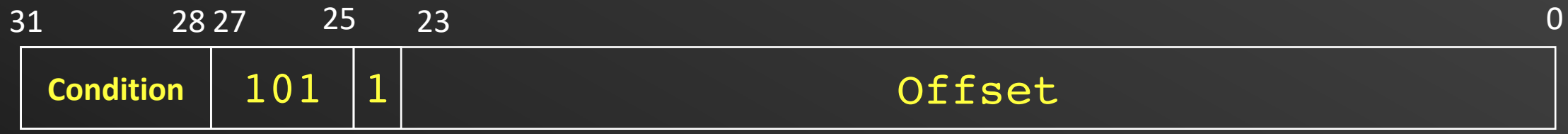
`BX: branch and exchange`



Calling and returning from a subroutine

Branch with Link (`BL`)

- **`BL`** used to make subroutine call
- Return address (BL inst. location+ 4) is stored in the link register (R14)



- We can also conditionally make a functional call (more of this later)

Branch with Link (`BL`)

- **`BL`** used to make subroutine call

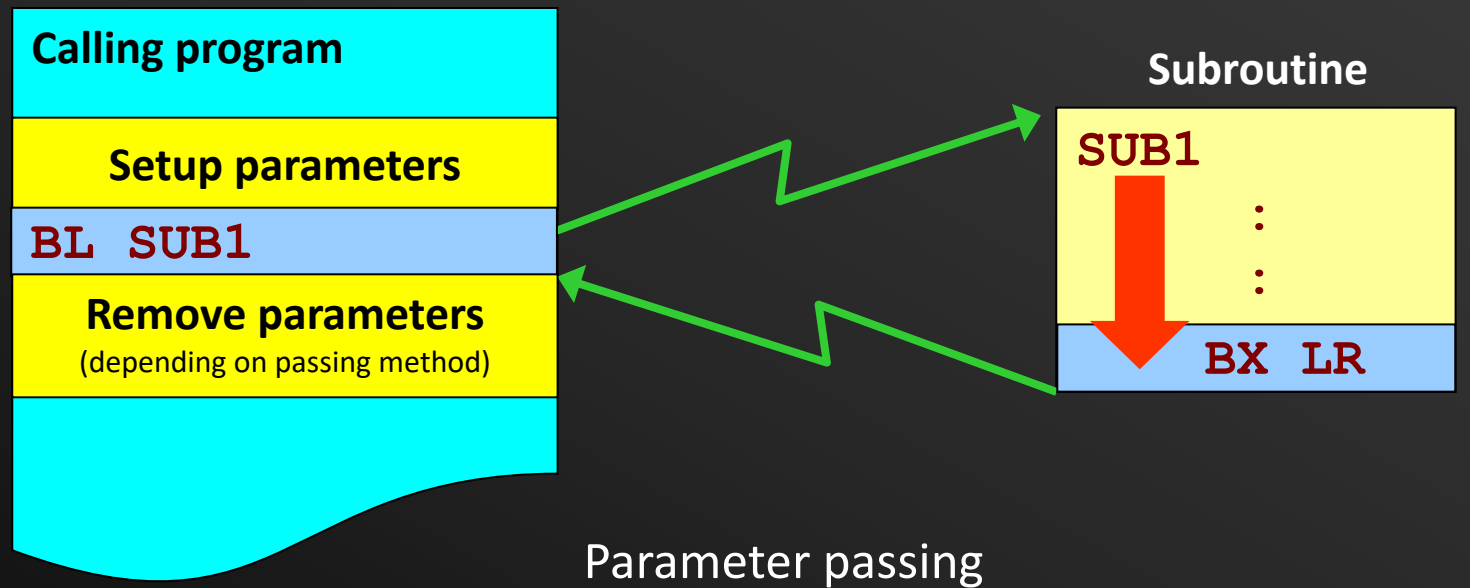
Subroutine Call

```
BL SortSub
```

- Execution sequence:
- Return address (BL inst. location + 4) is stored in the link register (R14)
- The subroutine address is stored to the PC

Parameter Passing

- Calling programs need to pass parameters to influence a subroutine's execution.
- Parameters must be setup properly before the subroutine is called and appropriately removed after returning.
- There are three basic methods to pass parameters, via **registers**, **memory block** or the **system stack**.



Parameter passing

Review Q1 – CALL

What register(s) **may be** affected by executing:

```
BL RegR1
```

(1) **PC**

(2) **PC** and **SP**

(3) **PC** and **LR**

(4) **PC, SP** and **LR**

Answer is 3

Review Q1 – CALL

What register(s) **may be** affected by executing:

```
BL RegR1
```

(1) **PC**

(2) **PC** and **SP**

(3) **PC** and **LR**

(4) **PC, SP** and **LR**

Answer is 3

Review Q2 – CALL

What value will be found at the **LR** after the execution of **`BL MYSUB`**?

```
0x000  MOV SP,#0xFFC
       :
0x010  BL MYSUB
       :
0x050  MYSUB ...
       :
```

(1) `0x100`

(2) `0x010`

(3) `0x014`

(4) `0x050`

Answer is 3

Review Q2 – CALL

What value will be found at the **LR** after the execution of **`BL MYSUB`**?

```
0x000  MOV SP,#0xFFC
       :
0x010  BL MYSUB
       :
0x050  MYSUB ...
       :
```

(1) `0x100`

(2) `0x010`

(3) `0x014`

(4) `0x050`

Answer is 3

Parameter Passing using Registers

- Parameters are placed into the register before calling the subroutine
- **Number** of parameters passed are **limited** to the **available registers**
  - Useful when number of parameters are **small**
  - Not all **R0-R12** are preferred to pass parameters

Parameter Passing using Registers

- Parameters are placed into the register before calling the subroutine
- **Number** of parameters passed are **limited** to the **available registers**
  - Useful when number of parameters are **small**
  - Not all **R0-R12** are preferred to pass parameters

Calling convention

R0-R3	R4-R11
Can be used to pass argument values Also used to return values from subroutine Subroutine can modify values	Used to hold local variables Not for passing arguments Must be preserved in the subroutine

Parameter Passing using Registers

- Parameters are placed into the register before calling the subroutine
- **Number** of parameters passed are **limited** to the **available registers**
  - Useful when number of parameters are **small**
  - Not all **R0-R12** are preferred to pass parameters

Calling convention

R0-R3
Can be used to pass argument values
to return values from subroutine
Subroutine can modify values

R4-R11
Used to hold local variables
Not for passing arguments
Must be preserved in the subroutine

R12: Scratchpad register, does not need to be preserved
Can be used sometimes as return register

Example: Bit Counting Subroutine

- Write a subroutine to:
  - Count the number of “1” bits in a word.
  - Return result in register **R0**.

- **Design considerations:**
- How do we transfer the word into the subroutine?
  - Put the word into a **register**, which can then be accessed within the subroutine (e.g. register **R1**).
- How do we check if each individual bit is a “1” or a “0”?
  - Rotate **R1** right 32 times with the carry bit. After each rotate, test carry bit to check if (**C**=1). If yes, increment bit counter register **R0**.

Review Q3 - Bit Counting Subroutine

- The **Count1s** subroutine:
  - Count the number of "1" bits in a word and return result in register **R0**.
- Passing parameter to **Count1s** subroutine:
  - Put the word into **register R1**, which can then be accessed within the subroutine.

```
Count1s  MOV     R0, #0          ;Clear R0
         MOV     R2, #32  ; Set counterR2 with 32
Loop     RRXS    R1,R1           ; Rotate right and extend R1
         ADC     R0,R0,#0        ; Add the carry to R0
         SUBS    R2,R2,#1        ; decrement counter by 1
         BNE     Loop            ; loop if not zero
         MOV     PC, LR          ; same as bx lr
```

BitCnt Question

Give the code to use **Count1s** to count the number of bits in the memory variable at address **0x100 (stored in R0)**.

```
Count1s  MOV     R0, #0        ;Clear R0
         MOV     R2, #32  ; Set counterR2 with 32
Loop     RRXS    R1,R1         ; Rotate right and extend R1
         ADC     R0,R0,#0      ; Add the carry to R0
         SUBS    R2,R2,#1      ; decrement counter by 1
         BNE     Loop          ; loop if not zero
         MOV     PC, LR        ; same as bx lr
```

(1)

```
MOV LR,PC
B  Count1s
```

(2)

```
BL Count1s
```

(3)

```
LDR  R1,[R0]
BL Count1s
```

(4)

```
MOV  R1,[R0]
B Count1s
```

Answer is 3

BitCnt Question

Give the code to use **Count1s** to count the number of bits in the memory variable at address **0x100 (stored in R0)**.

```
Count1s  MOV     R0, #0        ;Clear R0
         MOV     R2, #32  ; Set counterR2 with 32
Loop     RRXS    R1,R1         ; Rotate right and extend R1
         ADC     R0,R0,#0      ; Add the carry to R0
         SUBS    R2,R2,#1      ; decrement counter by 1
         BNE     Loop          ; loop if not zero
         MOV     PC, LR        ; same as bx lr
```

(1)

```
MOV LR,PC
B  Count1s
```

(2)

```
BL Count1s
```

(3)

```
LDR  R1,[R0]
BL Count1s
```

(4)

```
MOV  R1,[R0]
B Count1s
```

Answer is 3

Solution #2

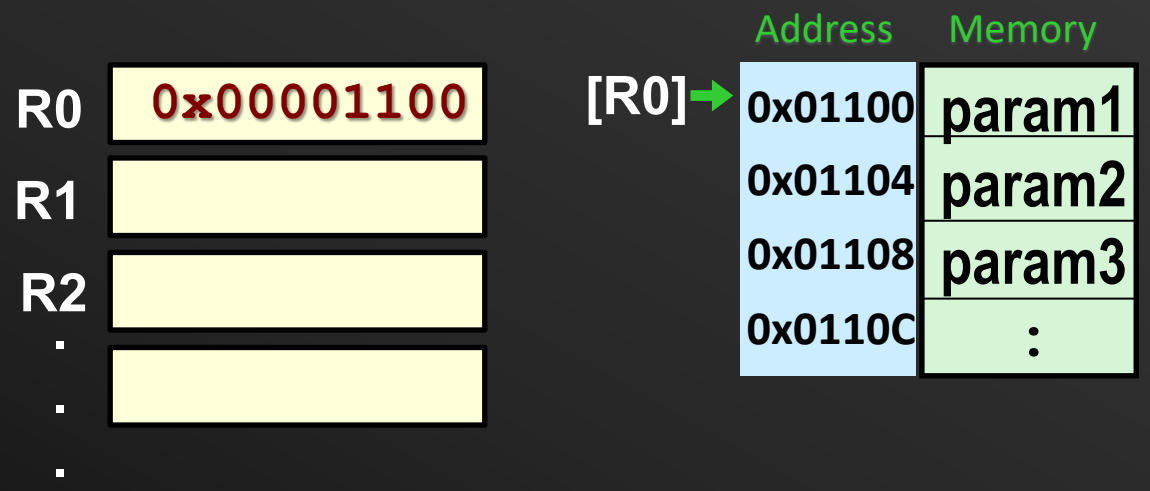
```
Count1s  EOR     R0,R0,R0      ;Clear R0
         ADD     R2, R0, #32   ; Set counterR2 with 32
         ADD     R3, R0, #1    ; Set R3 with 1
Loop     AND     R4, R3, R1, ROR R2          ; ?
         ADD     R0,R0,R4      ; Add the lsb of R0
         SUBS    R2,R2,#1      ; decrement counter by 1
         BNE     Loop          ; loop if not zero
         MOV     PC, LR        ; same as bx lr
```

Value passed in R1, return value in R0

VisUAL does not support `bx lr`

Parameter Passing using Memory

- A region in memory is treated like a mailbox and is used by both the calling program and subroutine.
  - Parameters to be passed are gathered into a **block** at a predefined memory location
  - The start address of the memory block is passed to the subroutine via an **address register**.
  - Useful for passing **large number of parameters**.



Example: Lower to Upper Case Subroutine

- Write a subroutine to:
  - To convert an ASCII string from lower to upper case.
  - The string is terminated by a NULL character (**0x00000000**).
  - The start address of the string is passed via **R0**.
  - A segment of the calling program shows how the parameter is setup and the subroutine called:

```
;Calling program
    :
MOV    R0,#0x100    ;move start addr. of string to R1
BL     Lo2Up        ;branch to Lo2Up subroutine
    :
```

Address	Memory
0x100	"a"
0x101	"p"
0x102	"p"
0x103	"l"
0x104	"e"
0x105	0x000
0x106	:

Passing params in memory

Given the problem to transfer lower-case to upper case, what is the right syntax to read each letter from memory to register R1 (base address is in R0)?

(1)
```
LDRB R1,[R0]
```

(2)
```
LDRB R1, [R0],#1
```

(3)
```
LDR R1,[R0],#1
```

(4)
```
LDR R1, [R0],#4
```

Answer is 2

Address	Memory
0x100	"a"
0x101	"p"
0x102	"p"
0x103	"l"
0x104	"e"
0x105	0x000
0x106	:

Efficient Memory Solution

```
Lo2Up   STMFD   SP!,{r0,r1}     ;save registers used within subroutine
Loop    LDRB    R1,[R0],#1      ;get current char from string in memory
        CMP     R1,#0           ;Compare with NULL
        BEQ     Done            ;if NULL char, branch to Done
        CMP     R1,#0x061       ;compare with lower limit "a"
        BLT     Loop            ;if smaller than "a", do not convert
        CMP     R1,#0x07A       ;compare with upper limit "z"
        BGT     Loop            ;if greater than "z" do not convert
        SUB     R1,R1,#32       ;convert to upper case by subtracting 32
        STRB    R1,[R0,#-1]     ;write modified char back to string in memory
        B       Loop            ;branch back to Loop
Done    LDMFD   SP!,{r0,r1}     ;restored saved registers before returning
        MOV     PC,LR           ;return from subroutine
```

Note: Subroutine modifies **R0** & **R1** but restores their original contents before returning. This produces a **transparent subroutine** that does not effect the proper operation of calling program